

Lecture 23: Machine Learning Continued

Heather Mattie, PhD

November 20, 2025

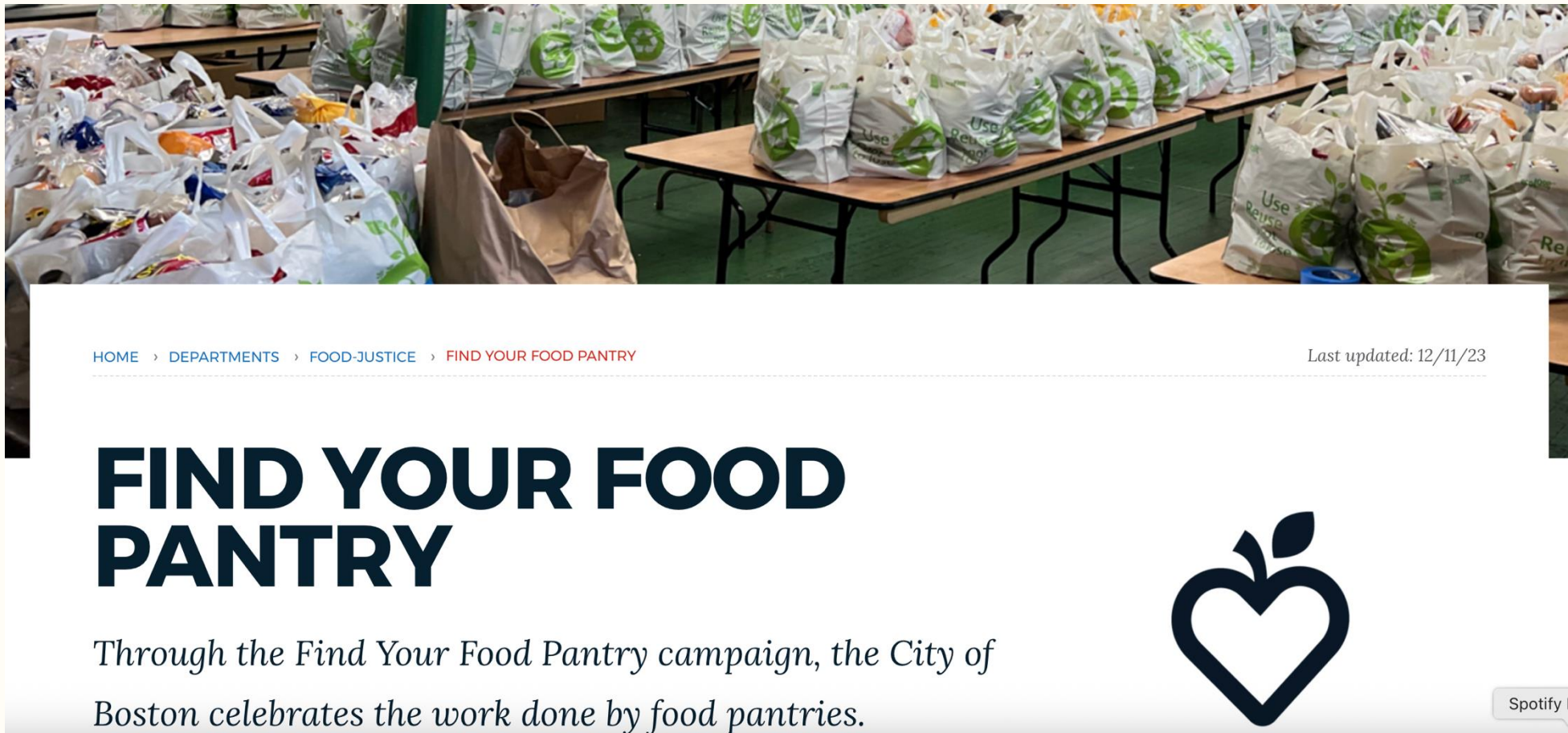


Recipe of the Day!

Budget-friendly Thanksgiving Recipes



City of Boston Find Your Food Pantry (free food and chances to volunteer)



Announcements

- Lab this week on machine learning
- Project preference form due Friday, November 21st
- Homework #4 due Friday, December 5th
- Homework #5 due Friday, December 12th
- No class November 25th and 27th



Introduction to Machine Learning

Logistic regression

Heather Mattie, PhD

Binary classification

Many public health decisions often hinge on yes/no outcomes that directly guide clinical or policy actions. This makes binary classification ML models very common and useful.

Many core public health outcomes are binary:

- Disease vs. no disease
- Hospitalized vs. not hospitalized
- Screen-positive vs. screen-negative
- Treatment success vs. failure

Binary classification

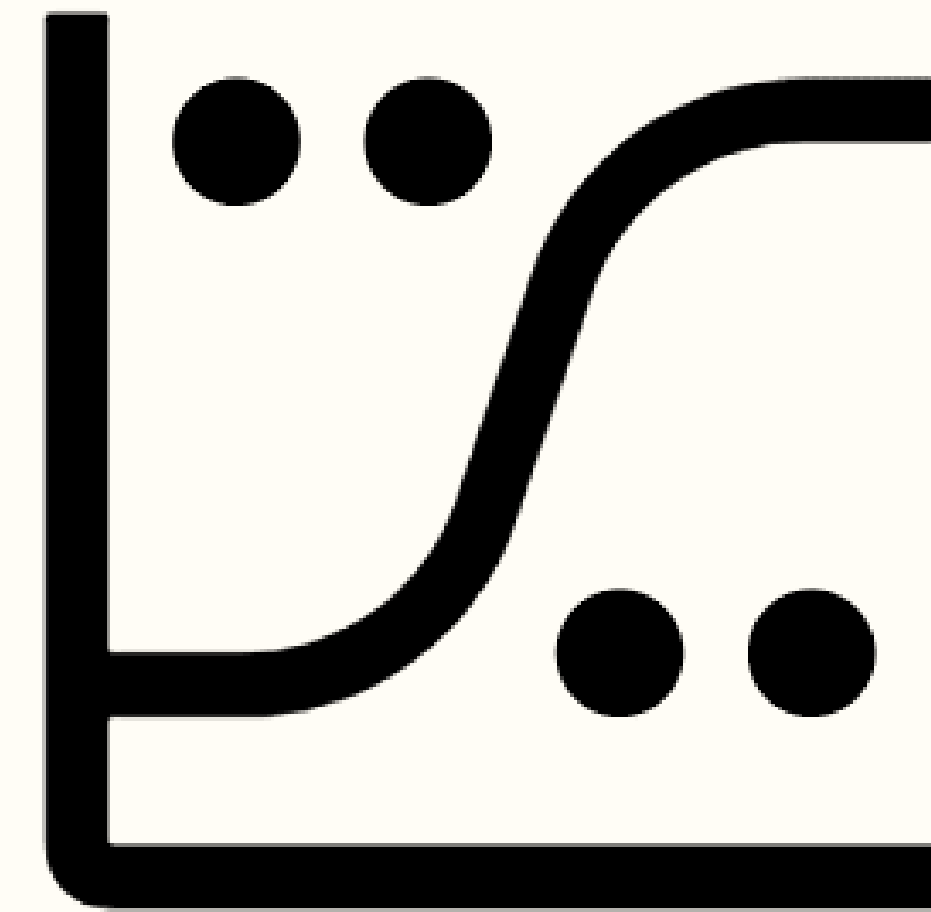
Binary classification ML models can:

- Support risk stratification and triage by identifying high-risk individuals.
- Enable early detection and prevention strategies (e.g., predicting diabetes, hypertension, HIV acquisition).
- Simplify communication of results to clinicians, policymakers, and patients.
- Work well with structured EHR/claims data, which often include categorical risk indicators.
- Facilitate evaluation using intuitive metrics like sensitivity, specificity, and AUC, aligned with clinical decision-making.

Logistic regression

Logistic regression is a foundational modeling approach for predicting binary outcomes, making it especially valuable in public health and clinical research.

- Models the probability that an event occurs:
 $p = P(Y = 1)$
- Predicts $Y = 1$ when $p >$ some threshold



Case study: melanoma

The **boot** package contains the **melanoma** dataset which records tumor thickness (mm) and survival status after tumor removal for 205 patients with malignant melanoma.

- Focus only on patients who either died from melanoma (1) or were still alive at the end of the study (2); drop all who died of other causes (3).

Case study: melanoma

```
library(tidyverse)
library(boot)
data(melanoma)

melanoma <- melanoma %>%
  filter(status %in% c(1, 2)) %>%
  mutate(status = as.factor(ifelse(status == 1, "Dead", "Alive")),
         y = as.factor(ifelse(status == "Dead", 1, 0)))
```

```
head(melanoma)
```

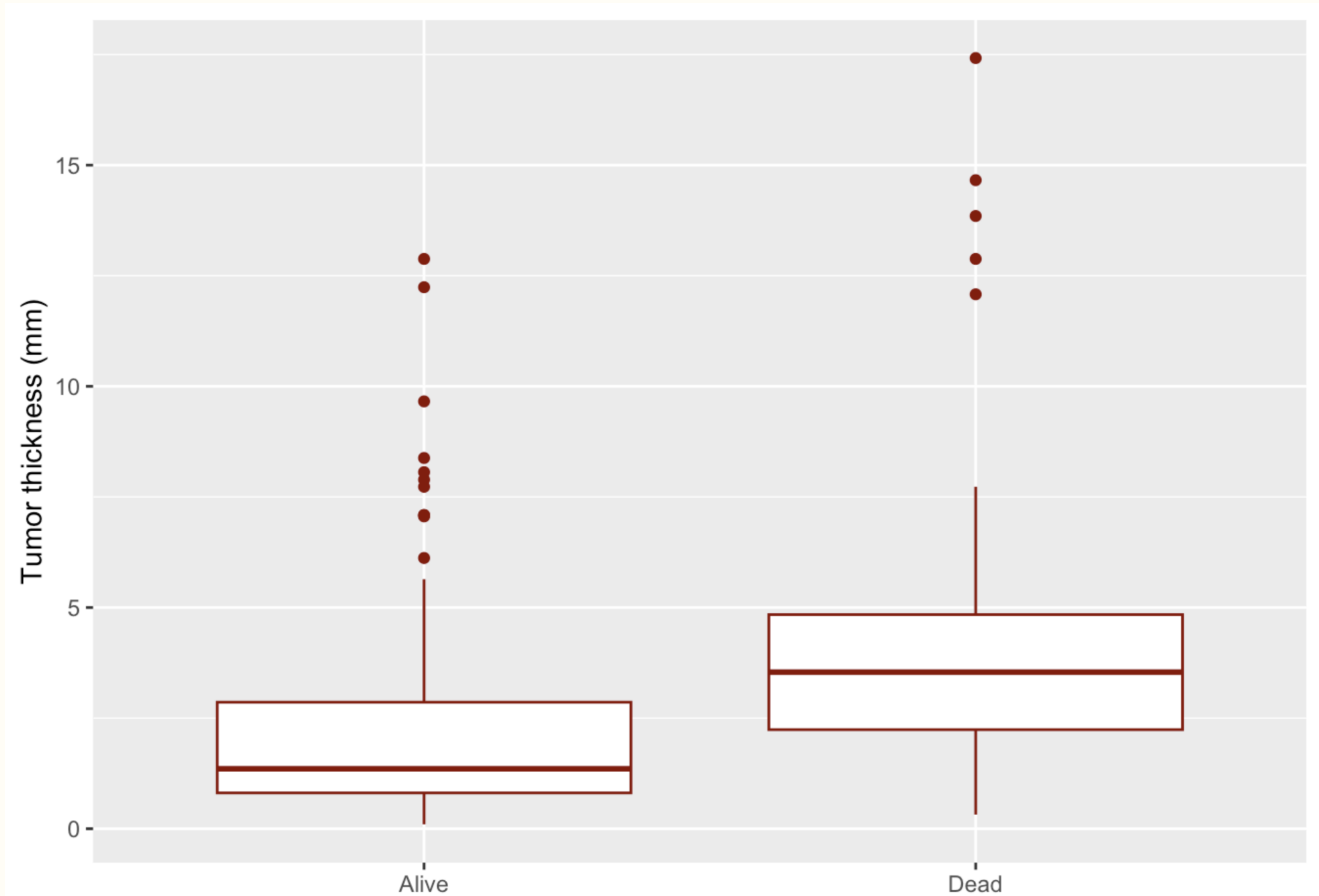
##	time	status	sex	age	year	thickness	ulcer	y
## 3	35	Alive	1	41	1977	1.34	0	0
## 5	185	Dead	1	52	1965	12.08	1	1
## 6	204	Dead	1	28	1971	4.84	1	1
## 7	210	Dead	1	77	1972	5.16	1	1
## 9	232	Dead	1	49	1968	12.88	1	1
## 10	279	Dead	0	68	1971	7.41	1	1

Case study: melanoma

```
table(melanoma$status)
```

```
##  
## Alive   Dead  
##   134    57
```

```
melanoma %>%  
  ggplot(aes(status, thickness)) +  
  geom_boxplot(color = "darkred") +  
  xlab("") +  
  ylab("Tumor thickness (mm)")
```



Training and test sets

There are more patients who lived than died in this dataset. This means there is **class imbalance** for the outcome of interest.

- Use the **stratified** function from the **splitstackshape** package to ensure the proportion of those who died and who lived is the same for both the training and test sets
- Use 70% of the data for training and 30% for testing

Training and test sets

```
library(splitstackshape)
library(caret)
set.seed(10)

x      <- stratified(melanoma, "y", 0.7, keep.rownames = TRUE)
train_set <- x %>% dplyr::select(-rn)
train_index <- as.numeric(x$rn)
test_set  <- melanoma[-train_index,]
```

Training and test sets

```
dim(train_set)
```

```
## [1] 134 8
```

```
head(train_set)
```

```
##      time status  sex  age  year thickness ulcer    y
##      <num> <fctr> <num> <num> <num>      <num> <num> <fctr>
## 1:  2492  Alive    0   54  1971    1.45    0    0
## 2:  3667  Alive    0   42  1967    3.22    0    0
## 3:  2431  Alive    0   49  1971    0.16    0    0
## 4:  3067  Alive    0   29  1969    0.81    0    0
## 5:  1652  Alive    0   58  1973    1.29    0    0
## 6:  5565  Alive    0   41  1962    2.90    0    0
```

```
dim(test_set)
```

```
## [1] 68 8
```

```
head(test_set)
```

```
##      time status  sex  age  year thickness ulcer  y
## 3      35  Alive    1   41  1977    1.34    0  0
## 5     185   Dead    1   52  1965   12.08    1  1
## 6     204   Dead    1   28  1971    4.84    1  1
## 7     210   Dead    1   77  1972    5.16    1  1
## 10    279   Dead    0   68  1971    7.41    1  1
## 11    295   Dead    0   53  1969    4.19    1  1
```



Model training

The **glm** function is used to fit a logistic regression model.

- Only using the training set to fit the model is what makes this a ML logistic regression model
- The **family = "binomial"** argument specifies logistic regression, rather than, e.g., Poisson regression

```
fit_logit <- glm(y ~ thickness,  
                 data = train_set,  
                 family = "binomial")
```

Making predictions

The **predict** function generates predicted probabilities for the test set using the model trained using the training set.

- The first argument is the model to use for prediction
- The second argument is the test set to use
- The **type = "response"** argument instructs the function to output predicted probabilities

```
p_hat_logit <- predict(fit_logit,  
                      newdata = test_set,  
                      type = "response")  
  
head(p_hat_logit)
```

```
##           3           5           6           7           10           11  
## 0.2076846 0.8534673 0.4186368 0.4412780 0.6019683 0.3737767
```

```
y_hat_logit <- ifelse(p_hat_logit > 0.5, "Dead", "Alive")  
  
head(y_hat_logit)
```

```
##           3           5           6           7           10           11  
## "Alive"  "Dead" "Alive" "Alive"  "Dead" "Alive"
```

Making predictions

Each predicted probability must then be transformed into either a 0 or a 1.

- This is based on a specific threshold – 0.5 is a common threshold
- A probability > 0.5 will be changed to a 1
- A probability ≤ 0.5 will be changed to a 0
- The threshold depends on the clinical context

```
p_hat_logit <- predict(fit_logit,  
                      newdata = test_set,  
                      type = "response")  
  
head(p_hat_logit)
```

```
##           3           5           6           7          10          11  
## 0.2076846 0.8534673 0.4186368 0.4412780 0.6019683 0.3737767
```

```
y_hat_logit <- ifelse(p_hat_logit > 0.5, "Dead", "Alive")  
  
head(y_hat_logit)
```

```
##           3           5           6           7          10          11  
## "Alive"  "Dead" "Alive" "Alive"  "Dead" "Alive"
```

The **caret** package

The **caret** package (**C**lassification **A**nd **RE**gression **T**raining) is a set of functions that streamline the process for creating predictive models. The package contains tools for:

- Data splitting
- Pre-processing
- Feature selection
- Model tuning using resampling
- Variable importance estimation



The confusion matrix

The **confusionMatrix** function from the **caret** package outputs the confusion matrix and several performance metrics for the model.

- **data**: the predicted labels
- **reference**: the actual ground truth labels
- **positive**: the value to use as the “positive” or “1” class

The confusion matrix

```
library(caret)
confusionMatrix(data = as.factor(y_hat_logit),
                 reference = test_set$status, positive = "Dead")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Alive Dead
##      Alive    43   18
##      Dead     2    5
##
##              Accuracy : 0.7059
##              95% CI : (0.5829, 0.8102)
##      No Information Rate : 0.6618
##      P-Value [Acc > NIR] : 0.2636727
##
##              Kappa : 0.2084
##
##      Mcnemar's Test P-Value : 0.0007962
```

```
##              Sensitivity : 0.21739
##              Specificity : 0.95556
##              Pos Pred Value : 0.71429
##              Neg Pred Value : 0.70492
##              Prevalence : 0.33824
##              Detection Rate : 0.07353
##      Detection Prevalence : 0.10294
##              Balanced Accuracy : 0.58647
##
##              'Positive' Class : Dead
##
```

The specificity is very high (95.6%), but the sensitivity is very low (21.7%). This means the model is better at predicting patients who will not die.

The confusion matrix and F_1 score

```
library(caret)
confusionMatrix(data = as.factor(y_hat_logit),
                 reference = test_set$status, positive = "Dead")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Alive Dead
##      Alive    43   18
##      Dead     2    5
##
##           Accuracy : 0.7059
##           95% CI : (0.5829, 0.8102)
##      No Information Rate : 0.6618
##      P-Value [Acc > NIR] : 0.2636727
##
##           Kappa : 0.2084
##
##      Mcnemar's Test P-Value : 0.0007962
```

```
##           Sensitivity : 0.21739
##           Specificity : 0.95556
##           Pos Pred Value : 0.71429
##           Neg Pred Value : 0.70492
##           Prevalence : 0.33824
##           Detection Rate : 0.07353
##           Detection Prevalence : 0.10294
##           Balanced Accuracy : 0.58647
##
##           'Positive' Class : Dead
##
```

```
cm$byClass["F1"]
```

```
##      F1
## 0.333
```

ROC

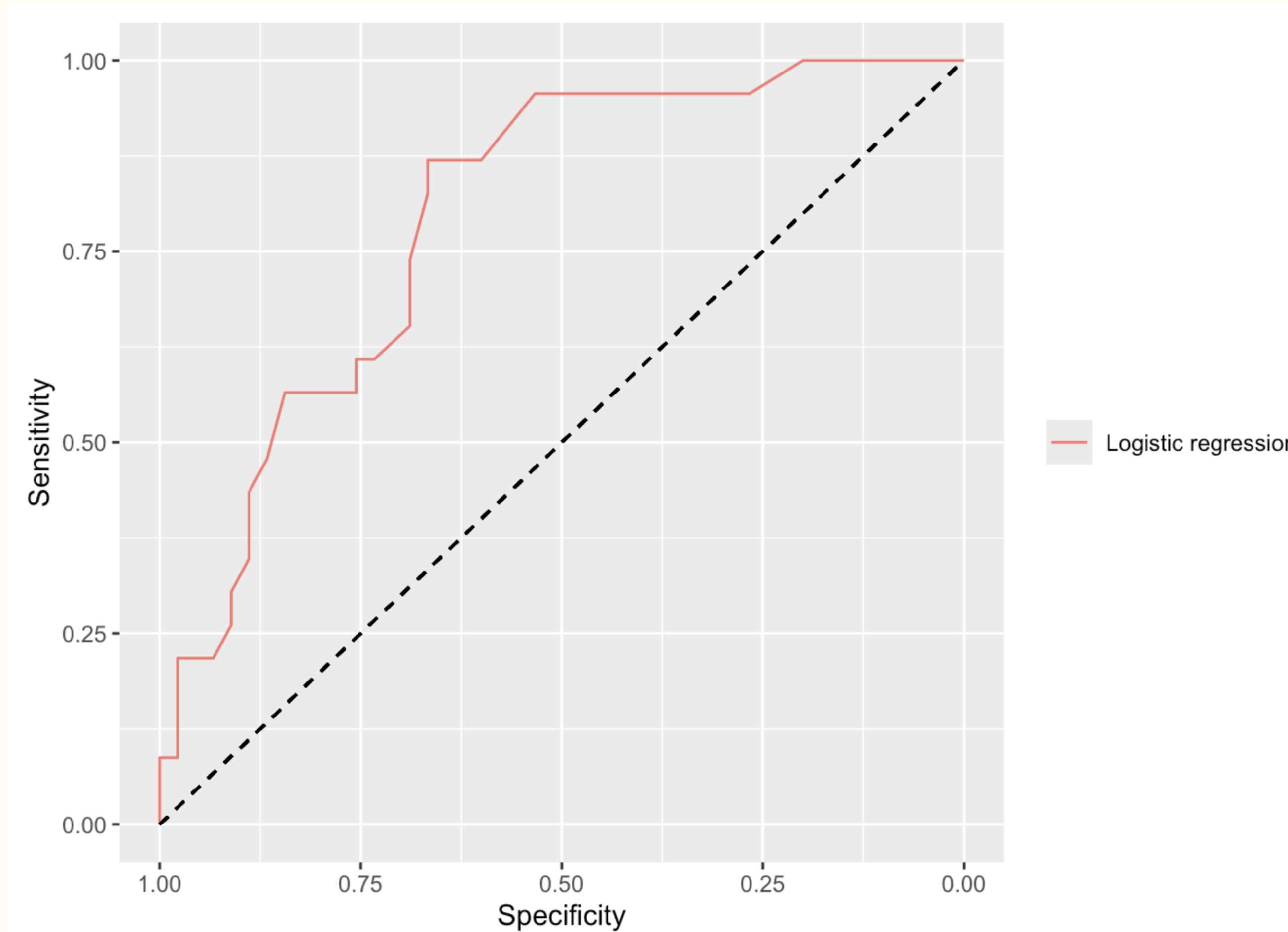
The ROC plot can be created using the **roc** and **ggroc** functions from the **pROC** package.

- The **roc** function calculates the sensitivity and specificity for various classification thresholds
- The **ggroc** function works like **ggplot** to plot the ROC curve

```
library(pROC)
roc_logit <- roc(test_set$y, p_hat_logit)
```

```
ggroc(list("Logistic regression" = roc_logit)) +
  theme(legend.title = element_blank()) +
  geom_segment(aes(x = 1, xend = 0, y = 0, yend = 1),
               color = "black",
               linetype = "dashed") +
  ylab("Sensitivity") +
  xlab("Specificity")
```

ROC



AUROC

The AUROC can be calculated using the **auc** function from the **pROC** package.

- This AUROC suggests the model has a reasonable ability to distinguish between individuals who died and those who lived
- A larger sample size might increase this value, especially given the class imbalance

```
auc(roc_logit)
```

```
## Area under the curve: 0.7942
```

Summary

- Logistic regression models the probability of a binary outcome using a logistic (sigmoid) link function.
- Outputs represent predicted probabilities, which are converted to classes using a decision threshold.
- Model performance can be assessed with confusion matrix–based metrics, ROC curves, and AUROC.
- Widely used in healthcare because it is interpretable, robust, and performs well with structured/tabular data.

Machine Learning Continued

Naïve Bayes

Heather Mattie, PhD

Naïve Bayes

Naïve Bayes is a machine learning algorithm that can be used for classification of 2 or more classes.

- Based on probability: *“Given these features, how likely is each class?”*
- Looks at how often each feature appears in each class in the training data
- Combines these pieces of evidence to choose the most likely class
- Called “**naïve**” because it assumes all features are independent (even though this is rarely true in practice)
- Despite the unrealistic assumption, it often performs very well, especially with many predictors

Case study: melanoma

```
library(tidyverse)
library(boot)
data(melanoma)

melanoma <- melanoma %>%
  filter(status %in% c(1, 2)) %>%
  mutate(status = as.factor(ifelse(status == 1, "Dead", "Alive")),
         y = as.factor(ifelse(status == "Dead", 1, 0)))
```

```
head(melanoma)
```

##	time	status	sex	age	year	thickness	ulcer	y
## 3	35	Alive	1	41	1977	1.34	0	0
## 5	185	Dead	1	52	1965	12.08	1	1
## 6	204	Dead	1	28	1971	4.84	1	1
## 7	210	Dead	1	77	1972	5.16	1	1
## 9	232	Dead	1	49	1968	12.88	1	1
## 10	279	Dead	0	68	1971	7.41	1	1

Training and test sets

```
library(splitstackshape)
library(caret)
set.seed(10)

x      <- stratified(melanoma, "y", 0.7, keep.rownames = TRUE)
train_set <- x %>% dplyr::select(-rn)
train_index <- as.numeric(x$rn)
test_set   <- melanoma[-train_index,]
```

Model training

The **naiveBayes** function from the **e1071** package can be used to train a Naïve Bayes ML model.

- The **type = "raw"** argument in the predict function returns predicted probabilities for each class
- Including **[,2]** retains the predicted probabilities for the 1 class (**[,1]** is the predicted probabilities for the 0 class)

```
library(e1071)
```

```
fit_nb <- naiveBayes(y ~ thickness, data = train_set)
```

```
p_hat_nb <- predict(fit_nb, test_set, type = "raw")[,2]  
head(p_hat_nb)
```

```
## [1] 0.155 1.000 0.345 0.390 0.804 0.271
```

```
y_hat_nb <- ifelse(p_hat_nb > 0.5, "Dead", "Alive")  
head(y_hat_nb)
```

```
## [1] "Alive" "Dead" "Alive" "Alive" "Dead" "Alive"
```

The confusion matrix and F_1 score

```
cm <- confusionMatrix(data = as.factor(y_hat_nb),
                      reference = test_set$status, positive = "Dead")
cm
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction Alive Dead
##      Alive    43   18
##      Dead     2    5
##
##              Accuracy : 0.706
##              95% CI : (0.583, 0.81)
##      No Information Rate : 0.662
##      P-Value [Acc > NIR] : 0.263673
##
##              Kappa : 0.208
##
##      McNemar's Test P-Value : 0.000796
```

```
##              Sensitivity : 0.2174
##              Specificity : 0.9556
##              Pos Pred Value : 0.7143
##              Neg Pred Value : 0.7049
##              Prevalence : 0.3382
##              Detection Rate : 0.0735
##      Detection Prevalence : 0.1029
##              Balanced Accuracy : 0.5865
##
##              'Positive' Class : Dead
##
```

```
cm$byClass["F1"]
```

```
##      F1
## 0.333
```



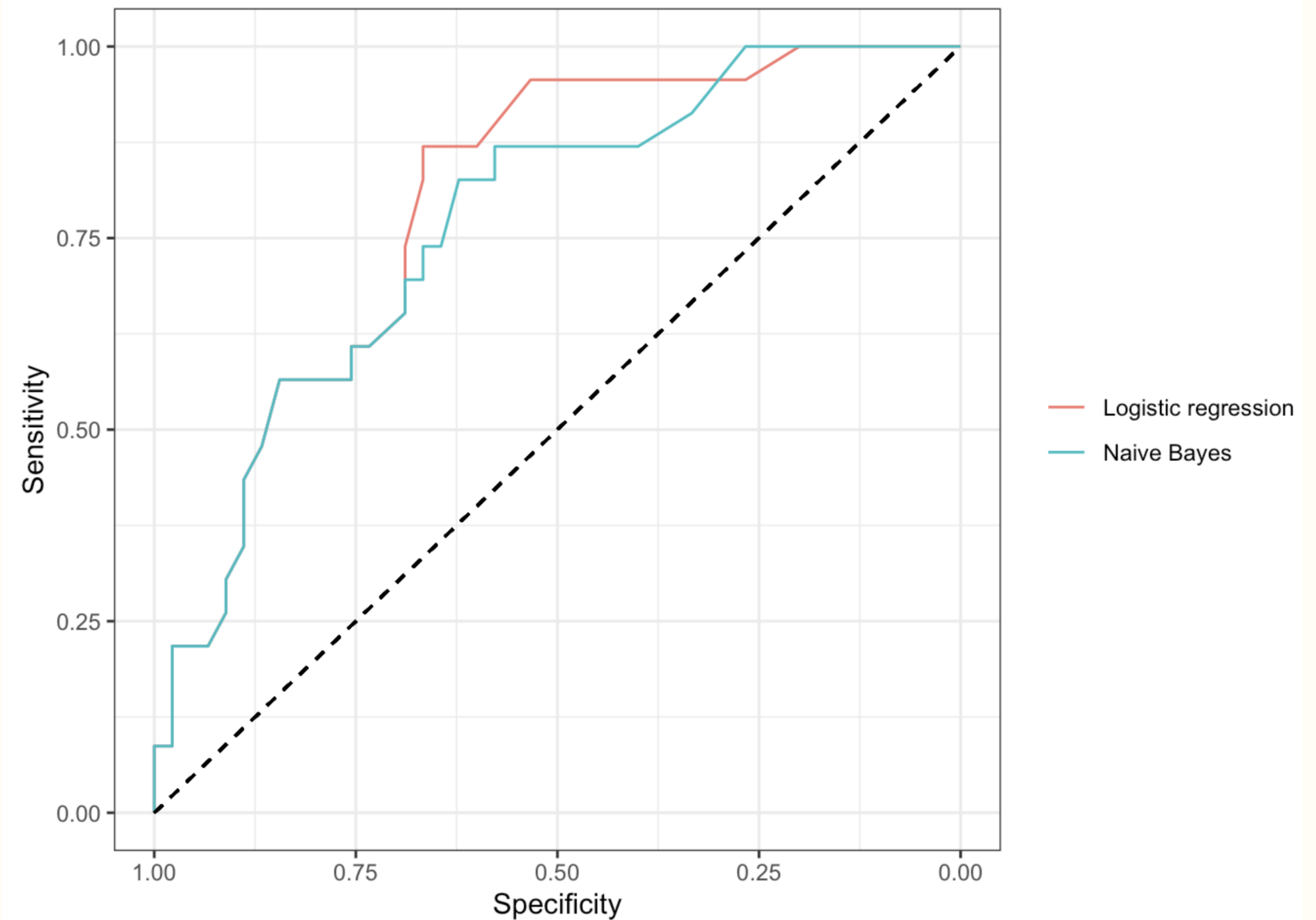
ROC and AUROC

```
roc_nb <- roc(test_set$y, p_hat_nb)

ggroc(list("Logistic regression" = roc_logit,
          "Naive Bayes" = roc_nb)) +
  theme(legend.title = element_blank()) +
  geom_segment(aes(x = 1, xend = 0, y = 0, yend = 1),
              color = "black",
              linetype = "dashed") +
  ylab("Sensitivity") +
  xlab("Specificity")
```

```
auc(roc_nb)
```

```
## Area under the curve: 0.768
```



Logistic regression vs Naïve Bayes

Algorithm	Overall accuracy	Sensitivity	Specificity	F ₁ score	AUROC
Logistic regression	0.706	0.2174	0.9556	0.333	0.794
Naïve Bayes	0.706	0.2174	0.9556	0.333	0.768

Note: identical confusion matrices reflect identical decisions at **one cutoff** (0.5), while different AUROC values reflect differences in the underlying predicted probabilities across **all cutoffs**.

- Looking at AUROC, the logistic regression model performs slightly better

Machine Learning Continued

Cross validation

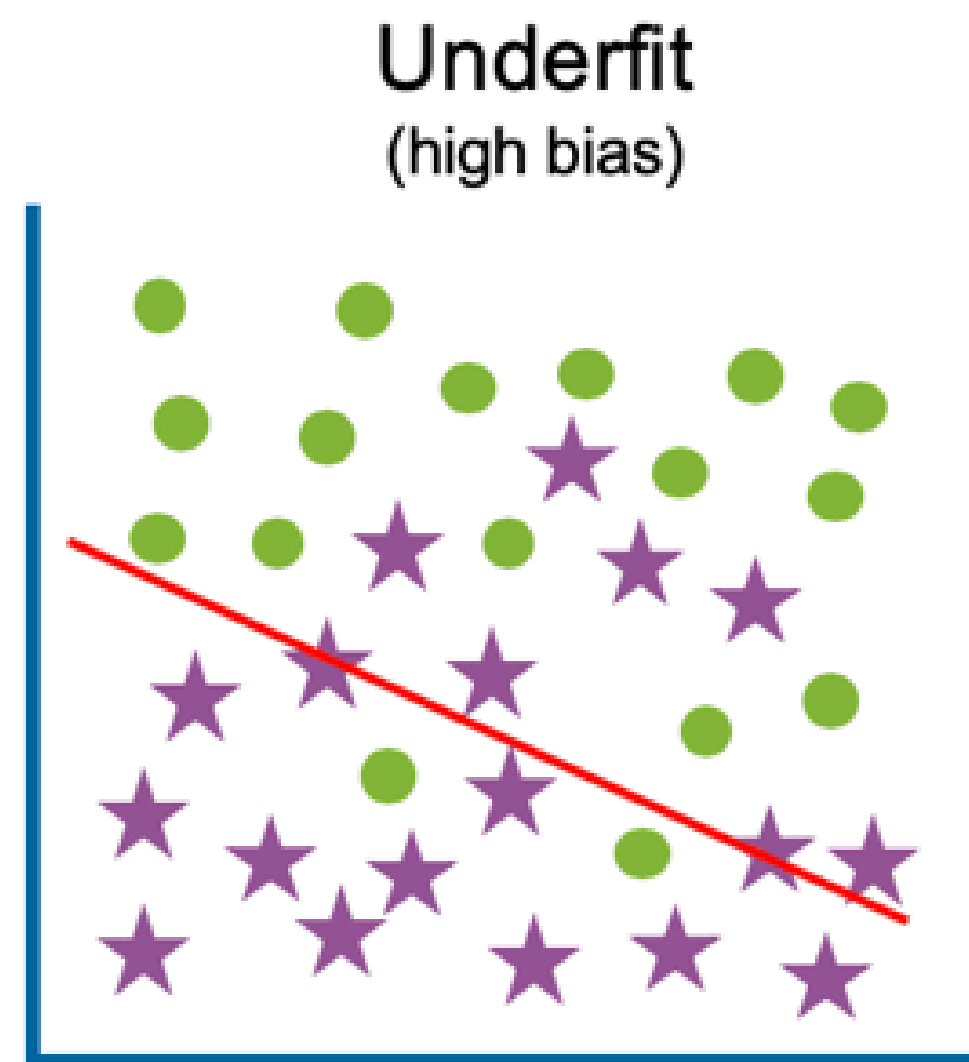
Heather Mattie, PhD

Overfitting

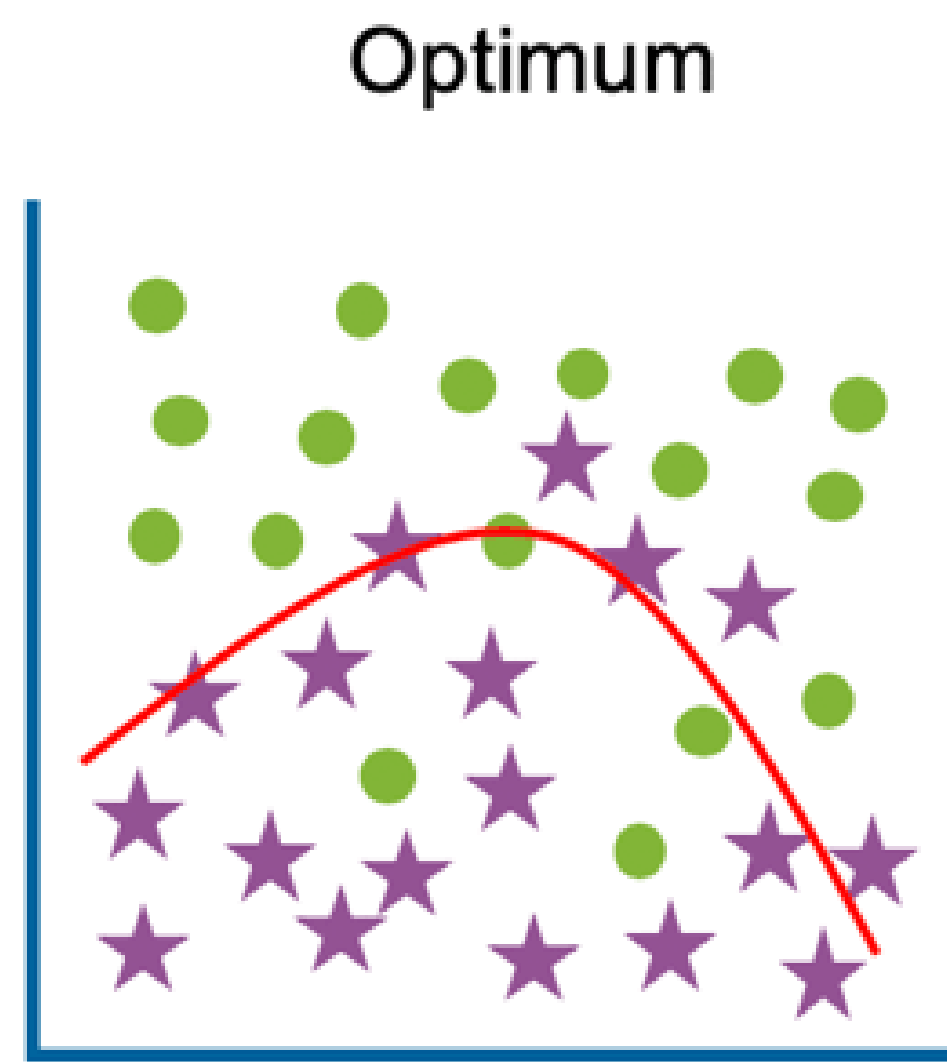
Overfitting is when a machine learning model learns the training data too well, including its noise and irrelevant details, leading to high accuracy on the training data but poor performance on new, unseen data.

- The model has become too complex and has memorized the training set rather than learning the general patterns.
- Leads to poor generalization, making predictions unreliable in real-world settings
- More likely with small datasets or highly flexible algorithms (e.g., deep trees, neural nets)
- Can be detected by comparing training vs. validation/test performance
- Prevented through cross-validation, regularization, simpler models, or more data

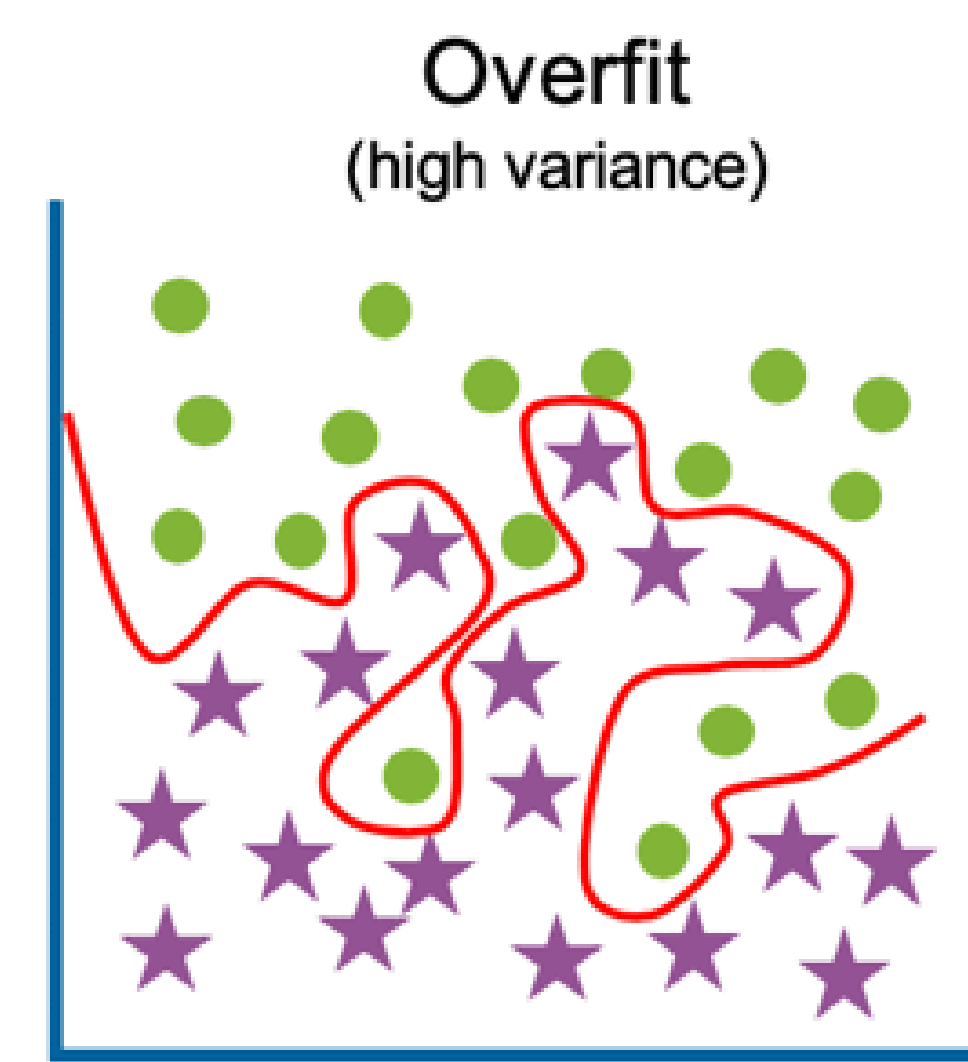
Overfitting



High training error
High test error



Low training error
Low test error

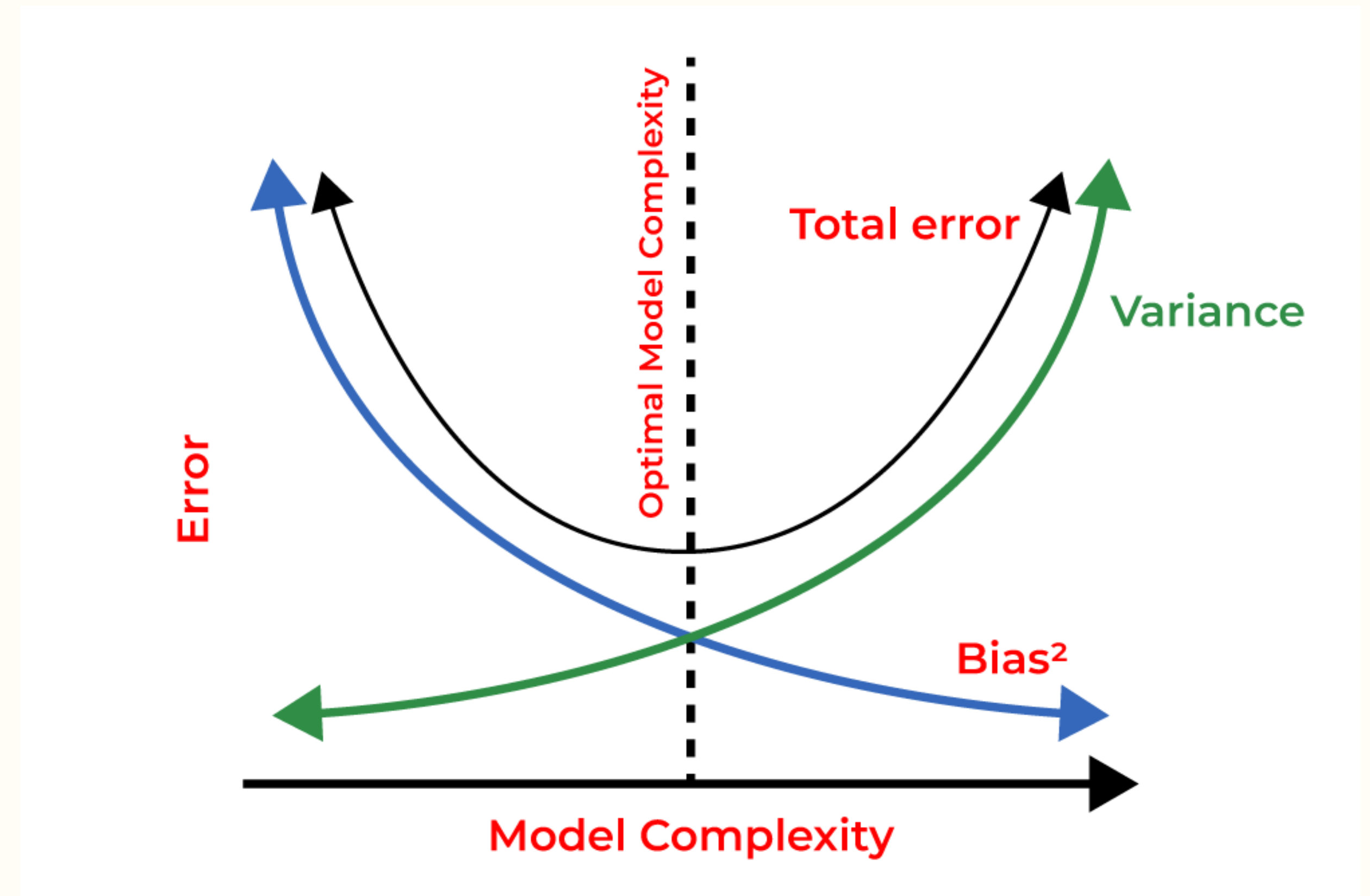


Low training error
High test error

Bias and variance

ML bias: the error that occurs when a model is **too simple** to capture the true patterns in the data.

ML variance: how much a model's predictions change when it's trained on different data.



Cross-validation

Cross-validation is a method for estimating how well a machine learning model will generalize to new, unseen data.

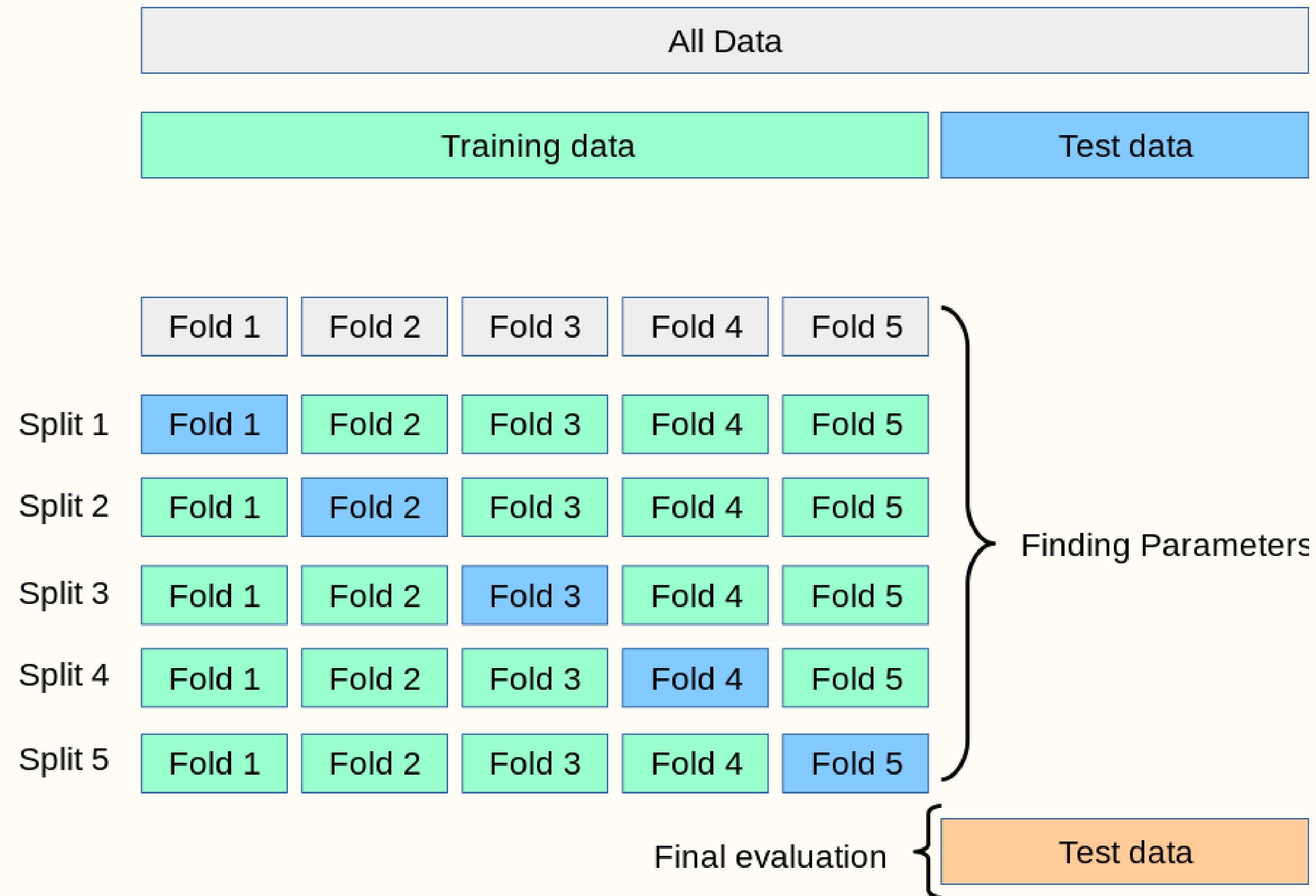
- Helps detect **overfitting** by testing the model on data it wasn't trained on
- Uses repeated train/test splits to get a more stable estimate of performance
- **Useful when the dataset is small**, since all data contribute to both training and evaluation
- Produces more reliable performance metrics than a single train/test split

k-fold cross-validation

With **k-fold cross-validation**, the dataset is divided into **k equal-sized folds (partitions)**. The most common values for k are 5 and 10.

- The model is trained k times, each time using a different fold as the test set
- Performance is averaged across all k runs, giving a stable estimate
- Ensures every observation is used for both training and testing
- Reduces variability in model performance estimates, especially on small datasets

k-fold cross-validation



Case study: melanoma

```
library(tidyverse)
library(boot)
data(melanoma)

melanoma <- melanoma %>%
  filter(status %in% c(1, 2)) %>%
  mutate(status = as.factor(ifelse(status == 1, "Dead", "Alive")),
         y = as.factor(ifelse(status == "Dead", 1, 0)))
```

```
head(melanoma)
```

##	time	status	sex	age	year	thickness	ulcer	y
## 3	35	Alive	1	41	1977	1.34	0	0
## 5	185	Dead	1	52	1965	12.08	1	1
## 6	204	Dead	1	28	1971	4.84	1	1
## 7	210	Dead	1	77	1972	5.16	1	1
## 9	232	Dead	1	49	1968	12.88	1	1
## 10	279	Dead	0	68	1971	7.41	1	1

trainControl

The **trainControl** function from the **caret** package is used to control the computational nuances and resampling procedures for the **train** function.

- Allows users to specify how models should be trained and evaluated, particularly concerning cross-validation and other resampling methods.

```
melanoma <- melanoma %>%  
  filter(status %in% c(1, 2)) %>%  
  mutate(status = as.factor(ifelse(status == 1, "Dead", "Alive")),  
         status = factor(status,  
                        levels = c("Alive", "Dead"),  
                        labels = c("Negative", "Positive")),  
         y = as.factor(ifelse(status == "Dead", 1, 0)))
```

```
set.seed(123)  
  
cv10 <- trainControl(  
  method = "cv",  
  number = 10,  
  classProbs = TRUE,  
  summaryFunction = twoClassSummary  
)
```

Logistic regression

```
log_cv <- train(  
  status ~ thickness,  
  data = train_set,  
  method = "glm",  
  family = binomial,  
  trControl = cv10,  
  metric = "ROC"  
)  
  
log_cv
```

```
## Generalized Linear Model  
##  
## 134 samples  
## 1 predictor  
## 2 classes: 'Alive', 'Dead'  
##  
## No pre-processing  
## Resampling: Cross-Validated (10 fold)  
## Summary of sample sizes: 120, 120, 121, 121, 121, 121, ...  
## Resampling results:  
##  
## ROC Sens Spec  
## 0.749 0.911 0.175
```

Logistic regression

```
p_hat_logit_cv <- predict(log_cv, test_set, type = "prob")[,2]

y_hat_logit_cv <- factor(ifelse(p_hat_logit_cv >= 0.5, "Dead", "Alive"),
                          levels=c("Alive","Dead"))

confusionMatrix(y_hat_logit_cv, test_set$status, positive = "Dead")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Alive Dead
##      Alive    43   18
##      Dead     2    5
##
##              Accuracy : 0.706
```

```
##           Sensitivity : 0.2174
##           Specificity : 0.9556
##           Pos Pred Value : 0.7143
##           Neg Pred Value : 0.7049
##           Prevalence : 0.3382
##           Detection Rate : 0.0735
##           Detection Prevalence : 0.1029
##           Balanced Accuracy : 0.5865
##
##           'Positive' Class : Dead
##
```

No change for this model compared to when k-fold cross-validation was not used.

Naïve Bayes

```
library(naivebayes)

nb_cv <- train(
  status ~ thickness,
  data = train_set,
  method = "naive_bayes",
  trControl = cv10,
  metric = "ROC"
)

nb_cv
```

```
## Naive Bayes
##
## 134 samples
## 1 predictor
## 2 classes: 'Alive', 'Dead'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 121, 121, 121, 120, 120, 121, ...
## Resampling results across tuning parameters:
##
## usekernel ROC Sens Spec
## FALSE 0.701 0.916 0.175
## TRUE 0.743 0.903 0.325
##
## Tuning parameter 'laplace' was held constant at a value of 0
## Tuning
## parameter 'adjust' was held constant at a value of 1
## ROC was used to select the optimal model using the largest value.
## The final values used for the model were laplace = 0, usekernel = TRUE
## and adjust = 1.
```

Naïve Bayes

```
p_hat_nb_cv <- predict(nb_cv, test_set, type = "prob")[,2]

y_hat_nb_cv <- factor(ifelse(p_hat_nb_cv >= 0.5, "Dead", "Alive"),
                      levels=c("Alive", "Dead"))

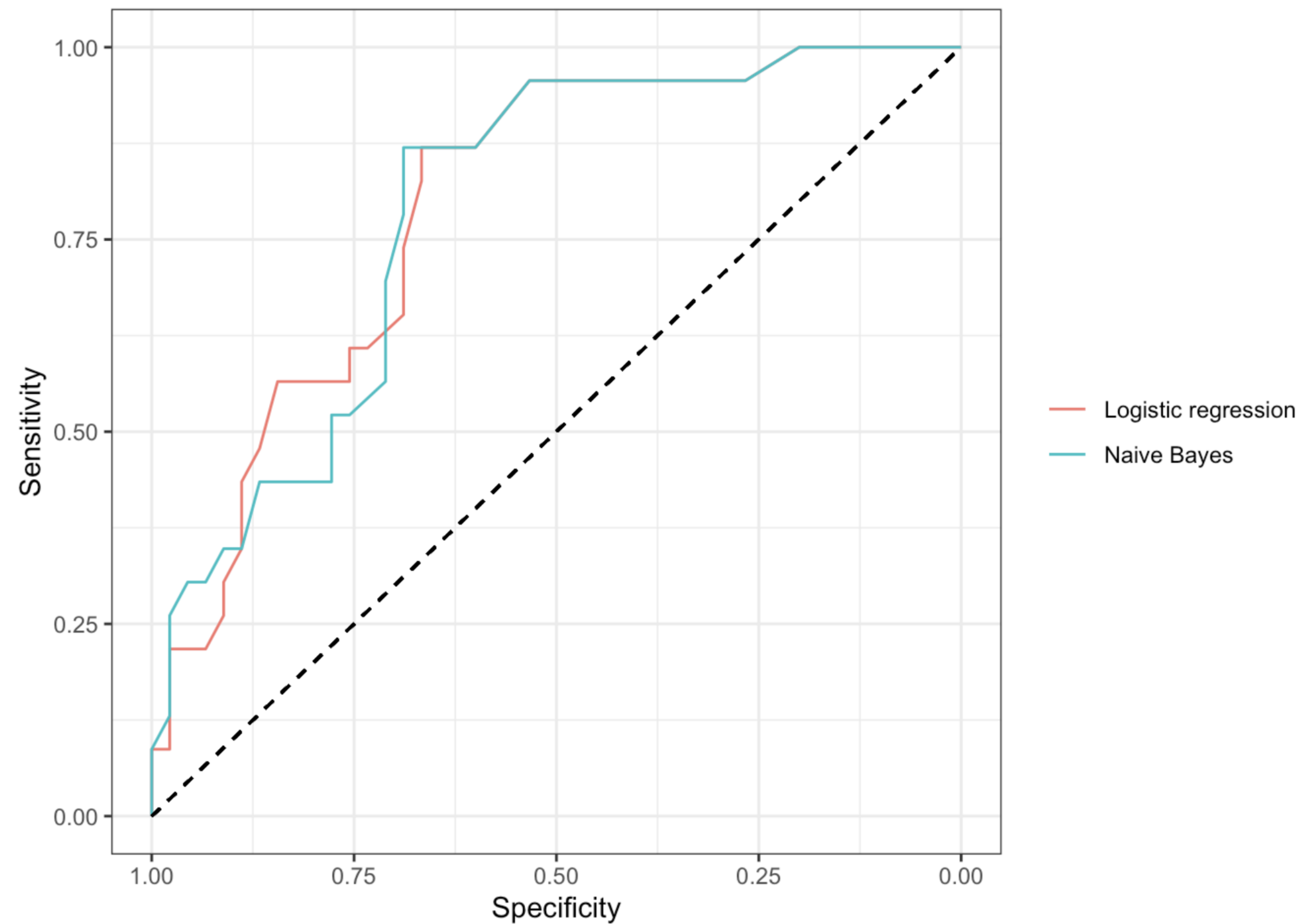
confusionMatrix(y_hat_nb_cv, test_set$status, positive = "Dead")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Alive Dead
##      Alive    40   15
##      Dead     5    8
##
##              Accuracy : 0.706
```

```
##              Sensitivity : 0.348
##              Specificity : 0.889
##              Pos Pred Value : 0.615
##              Neg Pred Value : 0.727
##              Prevalence : 0.338
##              Detection Rate : 0.118
##              Detection Prevalence : 0.191
##              Balanced Accuracy : 0.618
##
##              'Positive' Class : Dead
##
```

The overall accuracy did not change, but the sensitivity increased and the specificity decreased.

ROC and AUROC



```
auc(roc_logit_cv)
```

```
## Area under the curve: 0.794
```

```
auc(roc_nb_cv)
```

```
## Area under the curve: 0.788
```

Logistic regression vs Naïve Bayes

Algorithm	Overall accuracy	Sensitivity	Specificity	F ₁ score	AUROC
Logistic regression	0.706	0.2174	0.9556	0.333	0.794
Naïve Bayes	0.706	0.348	0.889	0.444	0.788

Choosing the best model is a little more difficult now. The Naïve Bayes model has the same overall accuracy, but a higher F₁ score and sensitivity, but a lower specificity. Overall, both models perform very similarly.